Lab 11: Visual Odometry

MCHA4400

Semester 2 2025

Introduction

In this lab, you will gain experience with a maximum likelihood solution to visual odometry that will serve as a foundation for the visual navigation aspects of Assignment 2.

GenAI Tip

You are strongly encouraged to explore the use of Large Language Models (LLMs), such as GPT, Gemini or Claude, to assist in completing this activity. If you do not already have access to an LLM, bots are available to use via the UoN Mechatronics Slack team, which you can find a link to from Canvas. If you are are unsure how to make the best use of these tools, or are not getting good results, please ask your lab demonstrator for advice.

1 Preparation

Tasks

a) Copy the most recent version of following files from previous labs (or Assignment 1) into your Lab 11 working directory:

- `src/Camera.h`
- `src/Camera.cpp`
- `src/rotation.hpp`
- `src/GaussianBase.hpp`
- `src/GaussianInfo.hpp`

b) Configure and build the `lab11` application.

c) Calibrate the DJI Mavic 2 camera by running the following command, which writes to `data/camera.xml` and displays a box visualisation for each calibration image:

Terminal

```
nerd@basement:~/MCHA4400/lab11/build$ ninja && ./lab11 -c ../data/config.xml
Calibrating camera
Chessboard: boardSize: [10 x 7], squareSize: 0.022
Scanning directory ./../data for file pattern "calibration.MOV"
Loading calibration.MOV...done, found 4855 frames
Reading calibration.MOV frame ???...done, detecting chessboard...found
Reading calibration.MOV frame ???...done, detecting chessboard...not found
 [etc.]
Reading calibration.MOV frame ???...done, detecting chessboard...found
Reading calibration.MOV frame ???...end of file found
Calibrating camera...done

Calibration data:
                  Bit flags: ???
               cameraMatrix: ???
                 distCoeffs: ???
              Focal lengths: (fx, fy) = (???, ???)
            Principal point: (cx, cy) = (???, ???)
  Field of view (horizontal): ??? deg
    Field of view (vertical): ??? deg
    Field of view (diagonal): ??? deg
      RMS reprojection error: ???
Display visualisation of calibration results
nerd@basement:~/MCHA4400/lab11/build$
```

d) Review the calibration results and sanity check against the manufacturer specifications.

Important

The success of the visual odometry result in this lab (and Assignment 2) depends greatly on the quality of the camera calibration.

2 DJI caption data

The Mavic drone stores flight and camera data as captions within a `SRT` subtitle file. The caption data provides useful information such as the altitude, longitude and latitude of the drone.

Tasks

a) Remove the `doctest::skip` decorator from each `SCENARIO` in `test/src/DJIVideoCaption.cpp` to enable these unit tests and run `ninja` to verify that they *fail* due to the incomplete implementations provided.

b) Within `src/DJIVideoCaption.cpp`, complete the implementation of `getVideoCaptions` to parse the `SRT` file and return a `std::vector` of `DJIVideoCaption`.

GenAI Tip

You can write a parser for the data in the `SRT` file by providing an LLM with a few examples of the data and the data structure you want to populate (i.e., `src/DJIVideoCaption.h`) for each frame of the video. Since an SRT is a fixed-format data file, this can be achieved elegantly using the `std::fscanf` function provided by including the standard C library

header `<cstdio>`.

c) Run `ninja` and ensure that all tests within `test/src/DJIVideoCaption.cpp` pass.

3 Ground plane sky dome model

For outdoor aerial vehicles at sufficient altitude, a ground plane and sky dome may be assumed—rays below the horizon terminate on a ground plane and rays above the horizon terminate at infinity. This enables the use of planar homographies for performing visual odometry.

The homogeneous coordinates of a pixel in frame $k-1$ can be predicted forward to frame k using the following homography:

$$\hat{\mathbf{p}}_j[k] = \mathbf{H}_j(\boldsymbol{\eta}_k, \boldsymbol{\eta}_{k-1})\mathbf{p}_j[k-1], \tag{1}$$

where $\mathbf{H}_j$ is given by

$$\mathbf{H}_j(\boldsymbol{\eta}_k, \boldsymbol{\eta}_{k-1}) = \begin{cases} \mathbf{H}_z(\boldsymbol{\eta}_k, \boldsymbol{\eta}_{k-1}) & \text{if } \mathbf{e}_3^\mathsf{T}\mathbf{R}_c^n[k]\mathbf{K}^{-1}\mathbf{p}_j[k] > 0, \\ \mathbf{H}_\infty(\boldsymbol{\eta}_k, \boldsymbol{\eta}_{k-1}) & \text{otherwise,} \end{cases} \tag{2}$$

and $\mathbf{H}_z$ represents the homography for a point on the ground plane, given by

$$\mathbf{H}_z(\boldsymbol{\eta}_k, \boldsymbol{\eta}_{k-1}) = \mathbf{K}(\mathbf{R}_c^n[k])^\mathsf{T}\left(\mathbf{I} - \frac{(\mathbf{r}_{C/N}^n[k-1] - \mathbf{r}_{C/N}^n[k])\mathbf{e}_3^\mathsf{T}}{\mathbf{e}_3^\mathsf{T}\mathbf{r}_{C/N}^n[k-1]}\right)\mathbf{R}_c^n[k-1]\mathbf{K}^{-1}, \tag{3}$$

and $\mathbf{H}_\infty$ represents the homography for a point on the plane at infinity, given by

$$\mathbf{H}_\infty(\boldsymbol{\eta}_k, \boldsymbol{\eta}_{k-1}) = \mathbf{K}(\mathbf{R}_c^n[k])^\mathsf{T}\mathbf{R}_c^n[k-1]\mathbf{K}^{-1}. \tag{4}$$

The image flow likelihood for each flow vector can then be expressed as

$$p(\mathbf{p}_j[k], \mathbf{p}_j[k-1] \mid \boldsymbol{\eta}_k, \boldsymbol{\eta}_{k-1}) = \mathcal{N}\left(\mathbf{r}(\mathbf{p}_j[k]); \mathbf{r}(\hat{\mathbf{p}}_j[k]), \sigma^2\mathbf{I}_{2\times 2}\right), \tag{5}$$

where we assume zero-mean isotropic Gaussian noise with standard deviation σ and $\mathbf{r}\colon \mathbb{P}^2 \to \mathbb{R}^2$ is defined by the following mapping:

$$\mathbf{r}\colon \begin{bmatrix} u \\ v \\ w \end{bmatrix} \mapsto \begin{bmatrix} \frac{u}{w} \\ \frac{v}{w} \end{bmatrix}. \tag{6}$$

Tasks

a) Within, `src/visual_odometry.cpp`, study the structure of the given function `runVisualOdometryFromVideo` and note the use of the `MeasurementOutdoorFlowBundle` class, including calls to the `costOdometry` and `predictedFeatures` member functions.

b) Within `src/MeasurementOutdoorFlowBundle.cpp`, implement the image flow measurement in the `MeasurementOutdoorFlowBundle` constructor, based on your Lab 10 solution. The constructor should populate all of the `protected` data members, as these will be used in the other member functions.

c) Within `src/MeasurementOutdoorFlowBundle.h`, implement `predictFlowImpl`, which returns the homogeneous coordinates of the predicted features in the current frame $\hat{\mathbf{p}}[k]$ (not taking into account lens distortion), given $\boldsymbol{\eta}_{k-1}$ and $\boldsymbol{\eta}_k$ extracted from the state vector $\mathbf{x}$ and the measured features in the previous frame in homogeneous coordinates $\mathbf{p}[k-1]$.

> **Note**
> The measured features in the current frame in homogeneous coordinates, $\mathbf{p}[k]$, are also provided, but these should only be used to classify the flow as above the horizon or below the horizon to select the appropriate sky-dome or ground plane homography to use, respectively.

d) Within `src/MeasurementOutdoorFlowBundle.h`, complete the implementation of `logLikelihoodImpl`, which calls `predictFlowImpl` and evaluates the log likelihood of the optic flow bundle.

e) Within `src/MeasurementOutdoorFlowBundle.cpp`, complete the implementation of `predictedFeatures`, which calls `predictFlowImpl` and then includes the effect of lens distortion to compute the predicted locations of features in the current image, $\hat{\mathbf{r}}^i_{Q/O}[k]$.

4 Visual odometry and visualisation

After completing the aforementioned tasks, you have most of the components necessary to complete a visual odometry experiment.

To see how it performs, we will create a visualisation with the following:

- Camera image.
- Predicted flow vectors drawn in blue (for both the inliers and outliers).
- Measured flow vectors drawn in green for inliers and red for outliers.
- Horizon drawn as a red curve.
- Ground grid over 2 km × 2 km area centred over initial position with North and East gridlines every 100 m drawn as black curves.
- Cardinal directions drawn as abbreviated text (N, NE, E, SE, S, SW, W, NW) on the horizon.
- Circular marker to indicate the translational velocity (orange).

> **Tip**
> Your estimator does not compute a translational velocity state, therefore you will need to approximate it. The epipolar point $\mathbf{e}[k]$ expresses a secant approximation to the translational velocity.

Tasks

a) Within `src/visual_odometry.cpp`, set the pose of the camera with respect to the body, $\mathbf{T}_c^b$, in `runVisualOdometryFromVideo` such that $\vec{b}_1 = \vec{c}_3$, $\vec{b}_2 = \vec{c}_1$ and $\vec{b}_3 = \vec{c}_2$ and assume that the body reference point B and the camera optical centre C coincide.

b) Within `src/visual_odometry.cpp`, complete the function called `getInitialPose`, which will return the initial pose $\boldsymbol{\eta}_0$ for the visual odometry experiment. Assume the camera optical axis is closely aligned with the $\vec{n}_1$ vector, such that it is initially pointing northwards.

c) Within `src/visual_odometry.cpp`, complete the implementations of `plotGroundPlane`, `plotHorizon`, `plotCompass` and `plotEpipole`.

d) Ensure the `lab11` program can run the visual odometry experiment by providing the following arguments

Terminal

```
nerd@basement:~/MCHA4400/lab11/build$ ninja && ./lab11 ../data/DJI_0218.MOV
[ninja-ing intensifies]
[passing tests]
Running visual odometry

Calibration data:
                   Bit flags: ???
                cameraMatrix: ???
                  distCoeffs: ???
              Focal lengths: (fx, fy) = (???, ???)
            Principal point: (cx, cy) = (???, ???)
  Field of view (horizontal): ??? deg
    Field of view (vertical): ??? deg
    Field of view (diagonal): ??? deg
Input video: ../data/DJI_0218.MOV
Subtitle file: ../data/DJI_0218.SRT
Total number of frames: 2180
Input video frame rate: 59.9401
Input video dimensions: [2688 x 1512]
After filtering by status, there are ??? associations.
No. inliers = ???, No. outliers  = ???
[optimisation step details]
CONVERGED: Newton decrement below threshold in ??? iterations
rBNn:
    ???
    ???
    ???
Rnb:
    ???    ???    ???
    ???    ???    ???
    ???    ???    ???
[continues for all frames]
nerd@basement:~/MCHA4400/lab11/build$
```

e) Export a video using

Terminal

```
nerd@basement:~/MCHA4400/lab11/build$ ninja && ./lab11 ../data/DJI_0218.MOV --export
```

and ensure the visualisation is similar to Figure 1.

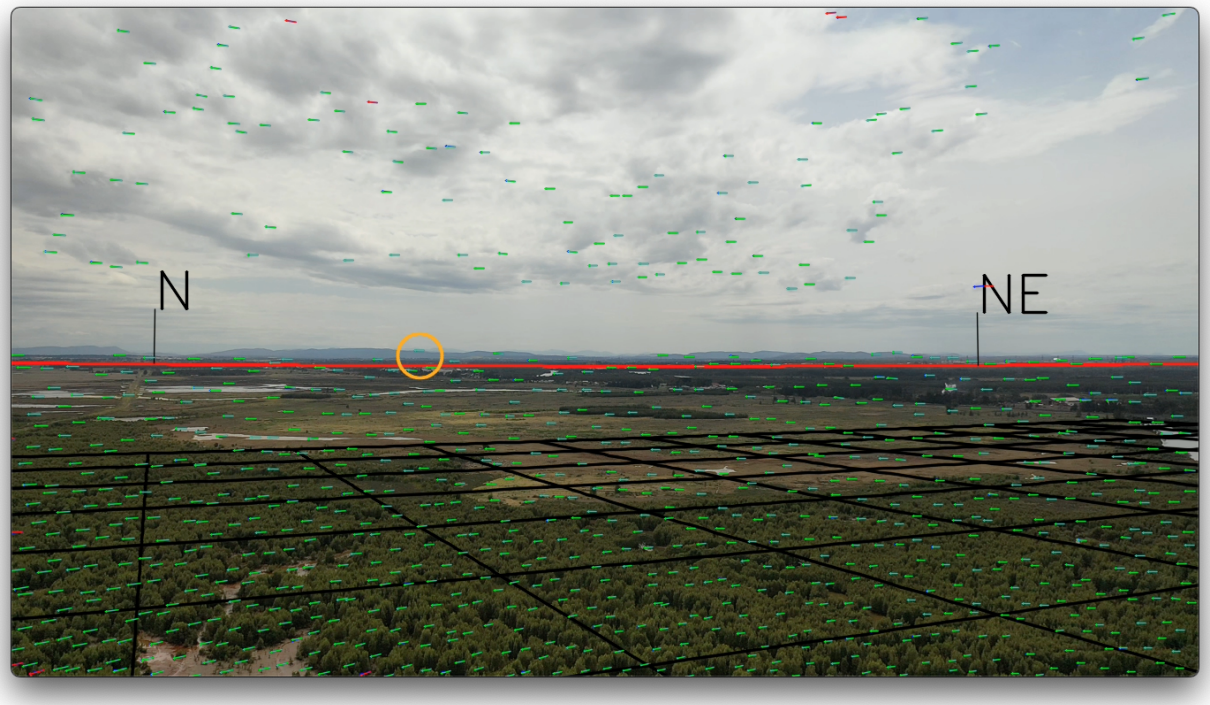

Figure 1: Visual odometry visualisation